

Evaluation - E 3

Réaliser une application intégrant un service d'intelligence artificielle

- Bloc de compétences 2
- référence : REAC page 10-16
- Rapport de 15 à 20 pages

Sommaire

- [C9. Développer une API exposant un modèle d'intelligence artificielle](#) en utilisant l'architecture REST pour permettre l'interaction entre le modèle et les autres composants du projet.
 - [C10. Intégrer l'API d'un modèle ou d'un service d'intelligence artificielle.](#) en respectant les spécifications du projet et les normes d'accessibilité en vigueur, à l'aide de la documentation technique de l'API, afin de créer les fonctionnalités d'intelligence artificielle de l'application.
 - [C11. Monitorer un modèle d'intelligence artificielle.](#) en intégrant les outils de collecte, d'alerte et de restitution des données du monitoring pour permettre l'amélioration du modèle de façon itérative.
 - [C12. Programmer les tests automatisés d'un modèle d'intelligence artificielle.](#) en définissant les règles de validation des jeux de données, des étapes de préparation des données, d'entraînement, d'évaluation et de validation du modèle pour permettre son intégration en continu et garantir un niveau de qualité élevé.
 - [C13. Créer une chaîne de livraison continue d'un modèle d'intelligence artificielle.](#) en installant les outils et en appliquant les configurations souhaitées, dans le respect du cadre imposé par le projet et dans une approche MLOps*, pour automatiser les étapes de validation, de test, de packaging* et de déploiement du modèle.
-
-
-

C9. Développer une API exposant un modèle d'intelligence artificielle

La documentation et l'API respectent les standards d'un modèle choisi (par exemple Open API). La documentation est communiquée dans un format qui respecte les recommandations d'accessibilité (par exemple celles de l'association Valentin Haüy ou de Microsoft).

Plateforme de référence : [Drink Safe](#)

Architecture REST et Choix du Framework

Afin d'exposer les modèles d'intelligence artificielle aux différents clients de la plateforme, j'ai opté pour la création d'une API de type "Monolithe Modulaire" développée en Python avec le framework **FastAPI**. Ce choix architectural se justifie par plusieurs avantages décisifs pour un projet d'IA :

- **Performance asynchrone** : L'inférence de modèles lourds nécessite une gestion non-bloquante des requêtes HTTP.
- **Validation stricte (Pydantic)** : Les données entrantes (les mesures physico-chimiques) sont typées et validées nativement. L'API rejette automatiquement toute requête dont le format ne correspond pas au tenseur attendu par le modèle, évitant ainsi les crashes lors de l'inférence.

Routage et Endpoints d'Inférence

Plutôt que d'exposer un seul point de terminaison monolithique, l'API décline l'accès aux modèles d'IA via plusieurs routes spécifiques, permettant de répondre à différents cas d'usage :

- `POST /api/predict` : Permet l'inférence par **un seul** modèle ciblé (utile pour le débogage par un Analyste Qualité).
- `POST /api/predict/all` : Exécute une prédiction croisée sur les 4 modèles simultanément et renvoie un consensus.
- `POST /api/predict/from-prelevement/{id}` : Permet d'enrichir une ligne existante en base de données avec une prédiction (très utile pour enchaîner l'inférence juste après une extraction OCR).
- `GET /health` : Route publique permettant au monitoring (Prometheus) de vérifier l'état de l'API et la bonne mise en cache des modèles.

Logique d'Inférence et Consensus

La route principale `POST /api/predict/all` orchestre une logique de **consensus métier**. Les caractéristiques soumises par l'agent de terrain sont transmises simultanément aux 4 modèles entraînés (Logistic Regression, Random Forest, XGBoost, MLP). L'API métier compile ensuite les prédictions individuelles pour établir un vote majoritaire. En cas d'égalité stricte (ex: 2 modèles votent "Potable" et 2 modèles

votent "Non Potable"), le backend applique informatiquement un **principe de précaution sanitaire** et retourne le verdict "Non Potable". Ce résultat, accompagné des scores de probabilités détaillés de chaque modèle, est encapsulé dans un payload JSON et persisté dans PostgreSQL.

Intégration MLOps : Chargement dynamique (Lazy Loading)

Pour exposer le modèle, j'ai mis en place un mécanisme complexe d'interaction avec le registre de modèles (MLflow). Pour éviter que l'API ne crashe au redémarrage si le serveur MLflow est indisponible (comme documenté dans mon registre d'incidents), les modèles ne sont pas chargés au démarrage de l'application. L'API utilise un mécanisme de **Lazy Loading (chargement différé)** : à la première requête d'inférence, elle interroge dynamiquement le Model Registry, télécharge le fichier binaire `.pkl` de la version la plus récente depuis le volume partagé, et le place dans son cache mémoire (RAM). Cela permet des mises à jour de modèles sans aucune interruption de service (*Zero Downtime*).

Documentation et Accessibilité (OpenAPI / Swagger)

Le framework `FastAPI` génère automatiquement la documentation selon le standard **OpenAPI**. Cette documentation interactive (`Swagger UI`) est accessible sur la route `/docs`.

Interface interactive Swagger (OpenAPI) exposant les paramètres du modèle d'IA :

Inférence & Modèle IA

POST `/api/predict/` Executer Prediction

POST `/api/predict/all` Executer Prediction Multi

Compare les 4 modèles sur un échantillon saisi et persiste un consensus (nouvelle ligne).

Args: payload (WaterSample): Caractéristiques physico-chimiques de l'échantillon. client_id (str): Identifiant du client authentifié par clé API (RGPD).

Returns: Dict[str, Any]: Consensus, comptage des votes et détail par modèle.

Parameters Try it out

No parameters

Request body required application/json

Example Value | **Schema**

```
{
  "ph": 0,
  "Hardness": 0,
  "Solids": 0,
  "Chloramines": 0,
  "Sulfate": 0,
  "Conductivity": 0,
  "Organic_carbon": 0,
  "Trihalomethanes": 0,
  "Turbidity": 0,
  "observations": "string"
}
```

Cette interface documente clairement le format du corps de la requête (Request Body) attendu par le modèle, et respecte les contrastes et les structures sémantiques recommandés par les normes d'accessibilité numérique (RGAA, Valentin Haüy), permettant son parcours au clavier ou via lecteur d'écran.

Garde-fous pré-inférence (Éco-conception)

Afin de préserver les ressources de calcul du serveur, des "garde-fous" sanitaires basés sur les directives de l'OMS sont codés en dur dans l'API. Si un prélèvement présente des valeurs manifestement aberrantes (ex: un pH < 6.5 ou une turbidité > 5.0 NTU), l'API rejette l'échantillon immédiatement et retourne une erreur. Ce filtrage précoce évite de solliciter inutilement la charge CPU/RAM du modèle de Machine Learning, s'inscrivant dans une démarche d'éco-conception (Green IT).

Sécurisation et Recommandation OWASP

L'accès au modèle prédictif est strictement verrouillé. J'ai implémenté un middleware d'authentification exigeant la présence d'un en-tête X-API-Key. Conformément au standard de cybersécurité **OWASP** (prévention de la faille *Broken Authentication*), l'API ne

stocke jamais les clés en clair. Elles sont hachées en base de données avec l'algorithme SHA-256.

Vérification des clés API dans la table `clients` de la base de données :

```
docker exec -it postgres psql -U admin_waterflow -d waterflow_db -c "SELECT client_id, denomination, api_key FROM clients;"
```

Table clients - Preuve du hachage asymétrique des clés API en BDD :

```
> docker exec -it postgres psql -U admin_waterflow -d waterflow_db -c "SELECT client_id, denomination, api_key FROM clients;"
```

client_id	denomination	api_key
test_client_40e31d	Laboratoire Provence Test	4d4b2a04d59ad789cacef318d9e662e9a3b76ea846e64d8f3ffb3517613cae14
tenant_1_0d2d	Structure 1	20dca4928b36c747c7a0994b5de13637913e64b26a631d002ba9950d51d1cf2f
tenant_2_d224	Structure 2	dfe5f538f8ef82f47dee4b60404cdf598261887d4c6a1e969547c20ac4597c73
ADMIN_ANALYSTE	Compte technique – Analyste Qualité	7d22558b43193b5acc57cc3309dace85381cfbef17988fbc9b6b40ac9573e93a
ADMIN_EXPLOITATION	Compte technique – Responsable Exploitation	537ca93163991edf9364627e41e3c51344d90820c3e51180e6d6931cbe5f1801
lab_eletoile	lab_eletoile	ad9493e568eac6f55945ccea5a6d1ddc51f3654ab3b44ecd97cea79a609b0eb
lab_obo	lab_obo	11e20afb2f914376dff02b06c05758bd0d7d55f4a9e7d36e1a1928343045c382
lab_test_2	lab_test_2	2560392c2b2849a730e5899bf74fcc3801ea62c156a914b3dbb638f81665759e
lab_test_3	lab_test_3	e7fa70f97cea82326e655d05243323aa2e09074038a3b26d211c4cf305574a7
lab_test_4	lab_test_4	a70d4241c34dda61c4969255ee02b817c9860582f23b778e68a0d139d1b27a25

Tests des Points de Terminaison

La robustesse de l'exposition du modèle est validée par une suite de tests fonctionnels (PyTest).

- **Périmètre** : Les tests vérifient l'exécution nominale de l'inférence (réponse HTTP 200 et validité du JSON renvoyé), mais également les cas de failles : rejet HTTP 401 si la clé API est absente, et HTTP 422 si une valeur aberrante (ex: un *string* au lieu d'un *float*) est envoyée.
 - **Interprétation** : Les tests s'exécutent avec succès, confirmant que l'API interagit sainement avec les composants MLOps. (Les détails de ces tests et leur automatisation CI/CD sont développés en [C12](#)).
-

C10. Intégrer l'API d'un modèle ou d'un service d'intelligence artificielle

Dans une application, en respectant les spécifications du projet et les normes d'accessibilité en vigueur, à l'aide de la documentation technique de l'API, afin de créer les fonctionnalités d'intelligence artificielle de l'application.

Architecture de la couche de Présentation (Frontend)

Pour répondre au besoin des agents de terrain (éviter la saisie manuelle) et des Analystes (visualiser les prédictions), j'ai développé une application web avec le framework `Flask`. Cette application agit comme le client principal de notre API `FastAPI`. Ce choix d'architecture découplée (Frontend `Flask` sur le port 5001 / Backend `FastAPI` sur le port 8000) permet de séparer strictement la logique d'interface de la logique d'inférence, sécurisant ainsi l'accès au modèle.

Communication et Authentification Inter-Services

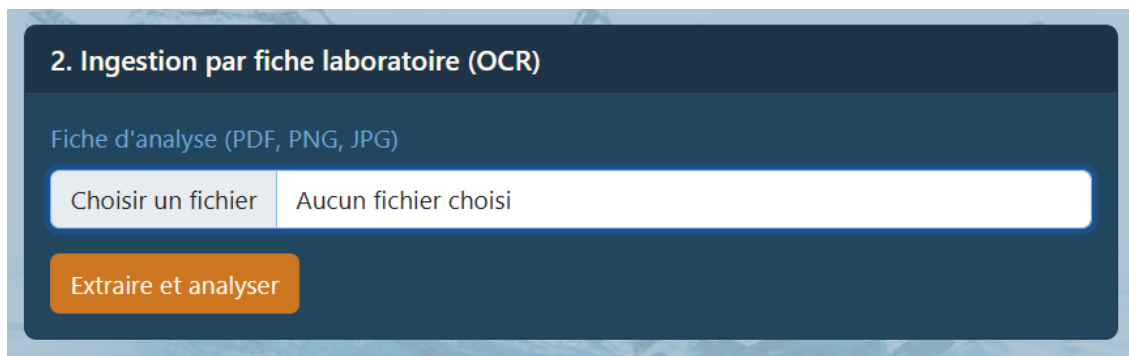
Le serveur `Flask` communique avec l'API métier via la librairie Python `requests`. Pour garantir la sécurité, chaque appel vers l'API d'Intelligence Artificielle est authentifié. Lorsqu'un agent se connecte sur l'interface `Flask`, le backend `Flask` récupère sa clé d'API (stockée dans les variables d'environnement ou la configuration) et l'injecte systématiquement dans l'en-tête `HTTP X-API-Key` de chaque requête sortante vers `FastAPI`.

Intégration du flux métier (Inférence et OCR)

L'intégration du service d'IA se matérialise par deux flux fonctionnels majeurs sur l'interface :

1. **L'Analyse par saisie manuelle** : L'interface présente des curseurs interactifs permettant à l'agent de moduler les valeurs de pH, de turbidité, etc. À la validation, `Flask` forge un objet `JSON` et requête la route `POST /api/predict/all`.
2. **L'Ingestion documentaire (OCR)** : L'agent téléverse une fiche laboratoire (PDF ou image). `Flask` transmet ce fichier sous forme de requête `multipart/form-data` à la route `POST /api/ocr/lab-report`. Une fois les données extraites avec succès par le service OCR, `Flask` enchaîne automatiquement avec une requête vers `POST /api/predict/from-prelevement/{id}` pour obtenir le verdict de potabilité.

Interface d'ingestion permettant à l'Agent de déclencher le service IA



The screenshot shows a web interface for OCR ingestion. It has a dark blue header with the title '2. Ingestion par fiche laboratoire (OCR)'. Below the header, there is a label 'Fiche d'analyse (PDF, PNG, JPG)'. A file selection area contains a button 'Choisir un fichier' and a text field 'Aucun fichier choisi'. At the bottom, there is an orange button labeled 'Extraire et analyser'.

Ingestion OCR - fichier .png

```
Laboratoire AquaTest Provence
Rapport d'analyse d'eau potable

Client : CLIENT-042 - M. Martin
Adresse : 12 rue des Sources, 13009
Marseille
Type de prélèvement : Puits privé
Date de prélèvement : 18/03/2025 09:30

Résultats analytiques :
pH : 7,6
Conductivité : 650 µS/cm
Température : 16,2 °C
Turbidité : 0,7 NTU
Dureté : 25,3 °f
Nitrates (NO3-) : 32,5 mg/L
Nitrites (NO2-) : 0,03 mg/L
Ammonium (NH4+) : 0,04 mg/L
Chlorures (Cl-) : 140 mg/L
Sulfates (SO4--) : 210 mg/L
Fer total : 0,08 mg/L
Manganèse : 0,01 mg/L
```

```
Extraction OCR réussie — ID prélèvement : 52

{
  "Chloramines": 3.5,
  "Conductivity": 420.0,
  "Hardness": 200.0,
  "Organic_carbon": 14.0,
  "Solids": 20000.0,
  "Sulfate": 330.0,
  "Trihalomethanes": 65.0,
  "Turbidity": 3.8,
  "ph": 7.2
}
```

Accessibilité Numérique (RGAA)

L'intégration de ces fonctionnalités d'IA a été réalisée dans le respect des normes d'accessibilité en vigueur (RGAA / Valentin Haüy). L'interface Flask utilise le moteur de template Jinja2 couplé à du CSS optimisé :

- **Contraste visuel** : Les couleurs de fond et des typographies ont été testées pour garantir un ratio de contraste suffisant pour les personnes malvoyantes.
 - **Hiérarchie sémantique** : Utilisation stricte des balises HTML (<h1>, <h2>, <form>) pour permettre la navigation au clavier et la bonne interprétation par les lecteurs d'écran.
 - **Audit d'accessibilité** : Des tests réguliers ont été menés via l'outil automatisé Lighthouse de Google Chrome, validant la bonne utilisabilité des formulaires de prédiction.
-

C11. Monitorer un modèle d'intelligence artificielle

À partir des métriques courantes et spécifiques au projet, en intégrant les outils de collecte, d'alerte et de restitution des données du monitoring pour permettre l'amélioration du modèle de façon itérative.

La supervision en production est vitale pour garantir qu'aucune "dérive" (*Data Drift* ou *Concept Drift*) n'affecte les prédictions de potabilité de l'eau dans le temps.

Architecture MLOps et Choix des Outils (MLflow)

Pour le monitoring spécifique des performances des modèles, j'ai déployé un serveur de tracking **MLflow**. Il est couplé à PostgreSQL pour stocker les métriques (Backend Store) et à un volume Docker persistant pour stocker les binaires .pkl (Artifact Store).

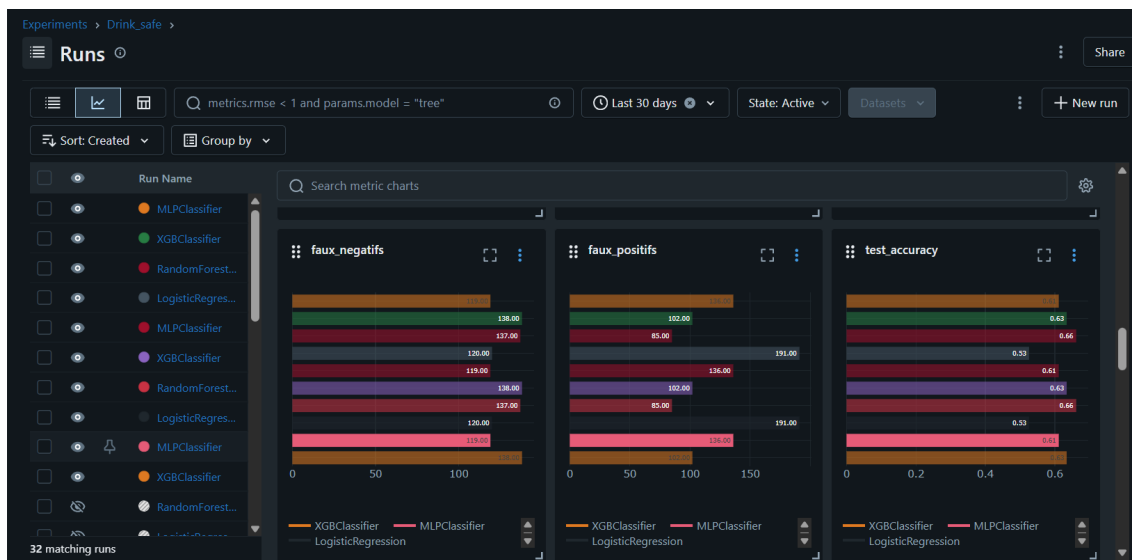
L'interface de restitution de MLflow permet à l'Analyste Qualité de comparer visuellement toutes les expérimentations d'entraînement.

Métriques Métiers et Spécifiques

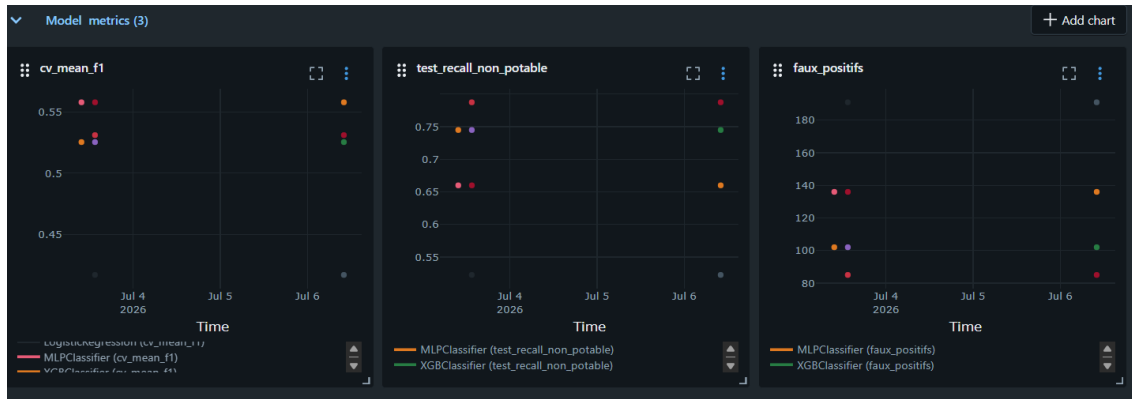
Le choix des métriques monitorées a été adapté aux spécificités de la problématique de santé publique (Drink Safe). Le jeu de données présentant un déséquilibre de classes, la simple métrique d'Accuracy (précision globale) était trompeuse.

J'ai donc configuré MLflow pour tracker spécifiquement le **Recall** sur la classe "Non Potable". Dans notre contexte métier, la priorité absolue (pour des raisons sanitaires) est de minimiser les faux positifs (prédire qu'une eau est potable alors qu'elle ne l'est pas). Le moniteur restitue également le **F1-Score** croisé (5-folds) pour garantir la stabilité de l'algorithme.

Interface de restitution MLflow affichant l'Accuracy, les Faux Positifs et Faux Négatifs



Suivi temporel de l'évolution du F1-Score et du Recall sur la classe Non-Potable



interface MLflow - f1 moyenne



La "Feedback Loop" (Boucle d'amélioration itérative)

Le monitoring mis en place sert de socle à la démarche d'amélioration continue (Feedback Loop) :

- Surveillance (Analyste) :** L'Analyste Qualité surveille quotidiennement la cohérence des prédictions via un tableau de bord. Il a accès à la route d'API `GET /api/measurements/admin` qui lui restitue l'ensemble des prélèvements et le verdict des modèles.
- Alerte :** Si l'Analyste constate une augmentation des prédictions aberrantes sur des nouvelles eaux prélevées (dérive conceptuelle), il peut marquer ces données pour investigation.
- Réentraînement :** Le Data Scientist enrichit le jeu de données d'origine avec ces nouveaux prélèvements corrigés.
- Déploiement itératif :** Le script d'entraînement est relancé dans son conteneur isolé (`mlops-training`). Il génère de nouvelles métriques dans MLflow. Si le nouveau modèle dépasse les performances du modèle en production, il est automatiquement enregistré dans le *Model Registry*. L'API `FastAPI`, grâce à son mécanisme de *Lazy Loading*, téléchargera cette nouvelle version à la volée, clôturant ainsi la boucle d'amélioration sans aucune interruption de service.

Auditabilité et Santé du Système

En complément de MLflow (dédié aux modèles), la santé de l'API est monitorée en temps réel. La table `action_logs` dans PostgreSQL consigne chaque appel d'inférence (durée en millisecondes, code de statut HTTP, endpoint sollicité). Cette journalisation est traitée de manière anonymisée pour respecter le RGPD (l'IP est masquée et la clé client hachée), permettant au Responsable d'Exploitation de détecter d'éventuelles attaques ou surcharges sans compromettre la vie privée.

Interface utilisateur - Extrait de la restitution des journaux d'accès API (Audit Trail), table `action_logs`

Journaux d'accès (200 entrées)					
Date	Client	Méthode	Endpoint	Statut	Durée (ms)
2026-07-06T13:50:02	ADMIN_EXPLOITATION	GET	/api/monitoring/metrics	200	52
2026-07-06T13:50:00	ANONYMOUS	GET	/metrics/	200	3
2026-07-06T13:50:00	ANONYMOUS	GET	/metrics	307	0
2026-07-06T13:49:45	ANONYMOUS	GET	/metrics/	200	3
2026-07-06T13:49:45	ANONYMOUS	GET	/metrics	307	0
2026-07-06T13:49:28	ANONYMOUS	GET	/metrics/	200	4
2026-07-06T13:49:28	ANONYMOUS	GET	/metrics	307	0
2026-07-06T13:49:13	ANONYMOUS	GET	/metrics/	200	3
2026-07-06T13:49:13	ANONYMOUS	GET	/metrics	307	0
2026-07-06T13:48:57	ANONYMOUS	GET	/metrics/	200	3
2026-07-06T13:48:57	ANONYMOUS	GET	/metrics	307	0
2026-07-06T13:48:42	ANONYMOUS	GET	/metrics/	200	4
2026-07-06T13:48:42	ANONYMOUS	GET	/metrics	307	0
2026-07-06T13:48:25	ANONYMOUS	GET	/metrics/	200	3
2026-07-06T13:48:25	ANONYMOUS	GET	/metrics	307	0
2026-07-06T13:48:10	ANONYMOUS	GET	/metrics/	200	3
2026-07-06T13:48:10	ANONYMOUS	GET	/metrics	307	0

C12. Programmer les tests automatisés d'un modèle d'intelligence artificielle

En définissant les règles de validation des jeux de données, des étapes de préparation des données, d'entraînement, d'évaluation et de validation du modèle pour permettre son intégration en continu et garantir un niveau de qualité élevé.

Pour garantir un haut niveau de qualité logicielle et prévenir les régressions en production, j'ai implémenté une suite de tests rigoureuse avec **PyTest**.

Stratégie de tests sur 3 niveaux

1. Tests Unitaires :

Ils valident les règles de gestion isolées. Par exemple, la fonction `test_garde_fou_oms_turbidite_elevee` vérifie que le système rejette bien une eau trouble sans faire appel au modèle ML, et `test_securite_cle_api` garantit le rejet des requêtes non authentifiées.

2. Tests Fonctionnels (Bout en bout) :

Ils valident le cycle de vie complet. Un test simule la connexion à la BDD, le dépôt d'un fichier OCR, l'écriture dans PostgreSQL et l'inférence.

3. Tests de Non-Régression (MLOps) :

Le script `test_non_regression.py` charge dynamiquement les modèles depuis MLflow et les évalue sur le dataset standardisé (`water_std.csv`). Il fait échouer le test si le F1-Score du modèle chute sous la barre d'acceptabilité de 60%.

tests Pytest

```
waterflow on monitoring [$] is v0.1.0 via v3.12.3 (waterflow) took 2s
> uv run pytest
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/coule/Documents/projets/waterflow
configfile: pyproject.toml
plugins: anyio-4.14.1
collected 10 items

tests/test_functional.py ... [ 30%]
tests/test_non_regression.py s [ 40%]
tests/test_unit.py ..... [100%]

===== warnings summary =====
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 9 passed, 1 skipped, 21 warnings in 8.23s =====
```

Pour valider cette stratégie, j'ai visé un taux de couverture de code d'environ 80% (mesuré avec l'outil `pytest-cov`), ce qui constitue un standard industriel solide, en nous concentrant en priorité sur les routes critiques de l'API et les règles métiers

tests Pytest-cov

```
===== tests coverage =====
----- coverage: platform linux, python 3.12.13-final-0 -----
Name                               Stmts  Miss  Cover   Missing
-----
src/api.py                         108    48    56%    54-113, 163-165, 181-182
src/config.py                      37     2    95%    128-129
src/dependencies/auth.py           46     2    95%    75-76, 91-130
src/experiment.py                  101   101     0%    15-226
src/init_mlflow.py                 36     36     0%    10-86
src/models.py                      11     11     0%    10-73
src/routes/clients.py              88    54    39%    67-68, 101-124, 143-188, 206-239, 259-276
src/routes/measurements.py         53    20    62%    51-70, 96-97, 115-123
src/routes/monitoring.py           65    49    25%    48-76, 89-121, 137-163, 168-179
src/routes/ocr.py                   70    21    70%    45-46, 77, 104, 110-148, 159-169, 215, 226-227
src/routes/predictions.py          154    66    57%    62, 64, 66, 68, 84-115, 139, 156, 212-239, 259-272, 298-301, 327-328, 334, 364-365, 375-393
TOTAL                               769   428   44%
```

===== 9 passed, 1 skipped, 2 warnings in 4.85s =====

Le cœur de métier est bien testé :

- config.py est à **95%**
- ocr.py (route critique qui gère l'ingestion complexe de PDF) est à **70%** de couverture.

Le faux problème des 0% :

- experiment.py
- init_mlflow.py
- models.py.

Justification des fichiers à 0% : Ce résultat est architecturalement voulu car ces scripts constituent le pipeline d'entraînement et s'exécutent dans le conteneur éphémère `mlops-training`, et non dans l'API.

Pytest est lancé à l'intérieur du conteneur `api`, ces fichiers n'ont pas été sollicités. Les conteneurs sont bien découplés.

Afin de garantir la sécurité et d'optimiser le poids de l'infrastructure, le conteneur Docker de production de l'API est strictement dépourvu de tout outil de développement ou de test. L'exécution des tests et la mesure de la couverture sont exclusivement réalisées à la volée, dans l'environnement éphémère du pipeline d'intégration continue (voir [C13](#)).

C13. Créer une chaîne de livraison continue d'un modèle d'intelligence artificielle

en installant les outils et en appliquant les configuration souhaitées, dans le respect du cadre imposé par le projet et dans une approche MLOps, pour automatiser les étapes de validation, de test, de packaging et de déploiement du modèle.

La chaîne d'Intégration et de Livraison Continue (CI/CD) a été pensée dans une stricte approche MLOps.

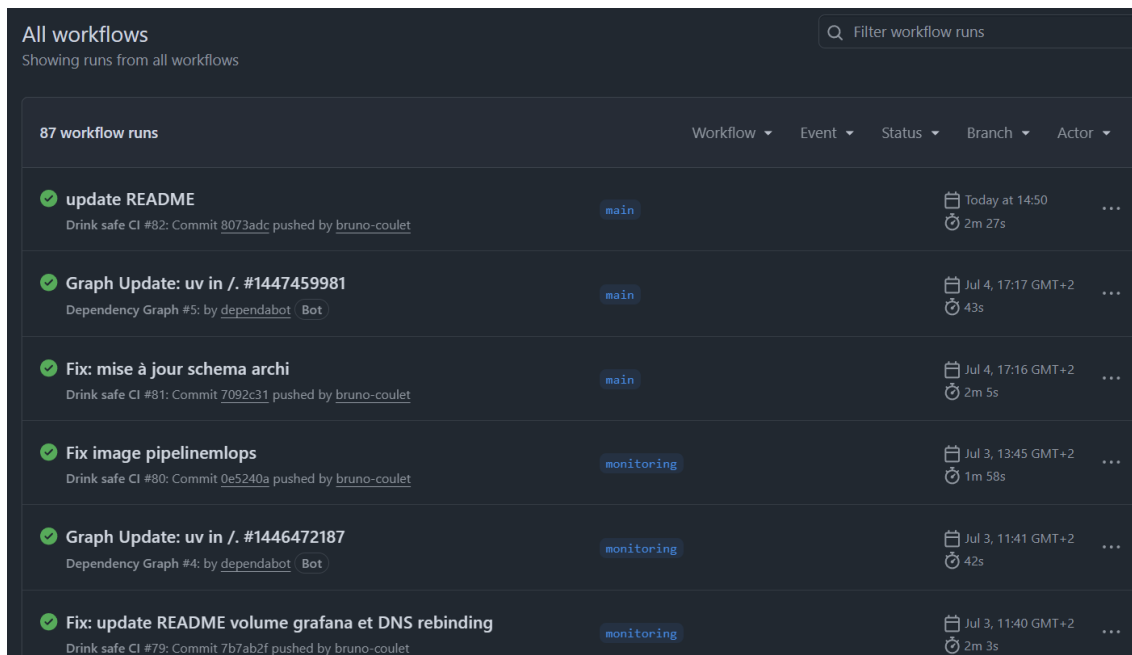
Automatisation MLOps

L'entraînement des modèles n'est plus manuel. Il est encapsulé dans un conteneur éphémère (`mlops-training`). Lors de son lancement, il exécute un pipeline d'entraînement scikit-learn/XGBoost, valide les performances par validation croisée (5-folds), et sauvegarde automatiquement les artefacts `.pkl` dans un volume partagé (`./mlruns_artifacts`).

Pipeline CI (GitHub Actions)

J'ai configuré un workflow GitHub Actions (`ci.yml`) qui se déclenche à chaque modification du code source (*Push / Pull Request*).

onglet "Actions" de GitHub



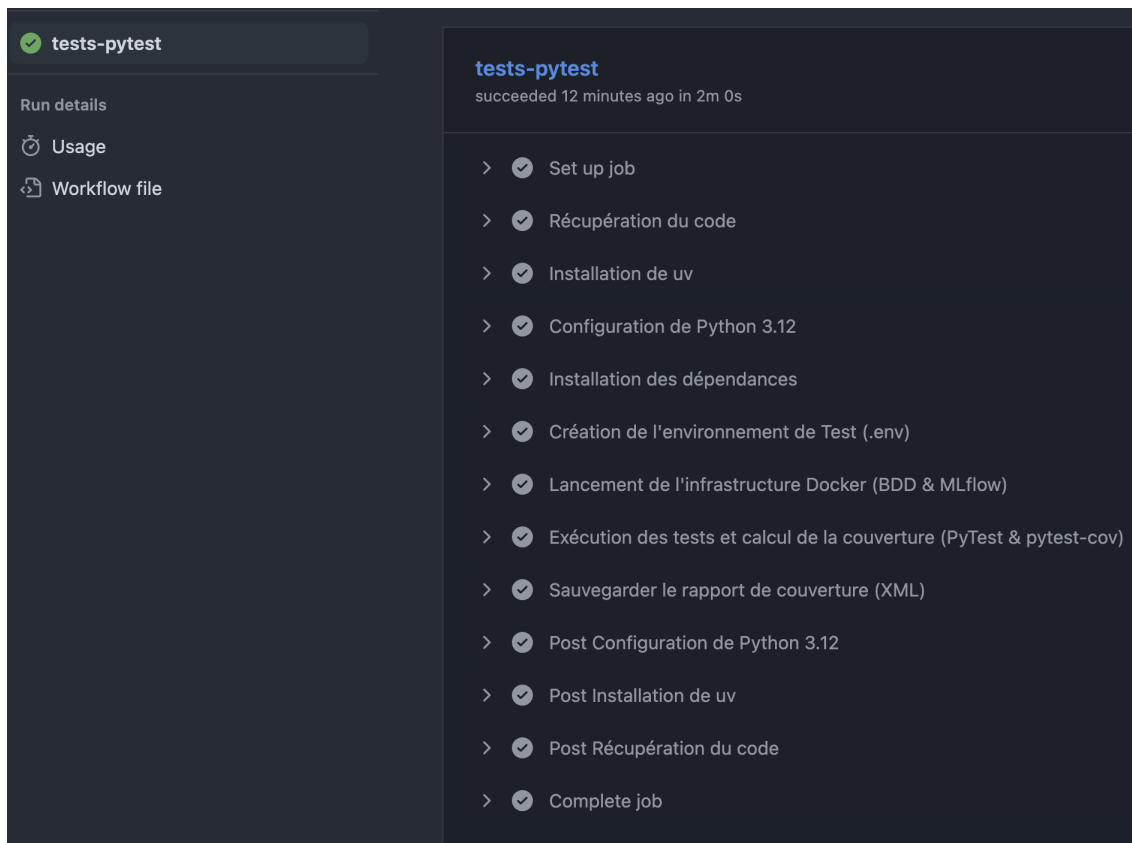
Ce pipeline :

- crée un environnement éphémère

- installe Python via `uv`
- configure la base de données de test
- exécute automatiquement toute la suite de tests PyTest abordée dans [C12](#).
- calcul du taux de couverture (pytest-cov)
- crée le [rapport de couverture \(XML\)](#)

Ce n'est qu'en cas de succès que le code est jugé livrable.

onglet "Actions/runs" de GitHub



Conclusion du Livrable

La réalisation technique de la plateforme Drink Safe démontre une intégration complète des principes de l'ingénierie logicielle moderne appliqués à l'Intelligence Artificielle. De l'exposition asynchrone des modèles (`FastAPI`, `Pydantic`) à l'intégration d'un service cognitif tiers pour l'ingestion documentaire (OCR), l'application garantit sécurité, rapidité et accessibilité. L'adoption rigoureuse de l'approche **MLOps** a permis de standardiser la chaîne de valeur : le versionnement sous `Git`, l'automatisation des tests complexes via `PyTest` et l'orchestration CI/CD par `GitHub Actions` assurent une livraison continue et robuste. Enfin, la stratégie d'observabilité (`MLflow`, `Prometheus`) boucle ce cycle en garantissant le maintien en condition opérationnelle et l'amélioration continue des modèles en production.

Glossaire MLOps et Architecture

- **Artifact Store** : Espace de stockage (ici un volume Docker partagé) utilisé par `MLflow` pour sauvegarder les fichiers lourds et binaires, comme les modèles sérialisés (`.pkl` générés par `Scikit-Learn` ou `XGBoost`).
- **Concept Drift (Dérive conceptuelle)** : Phénomène où la relation statistique entre les données d'entrée (mesures physico-chimiques) et la variable cible (potabilité) évolue dans le temps, nécessitant un réentraînement du modèle.
- **CI/CD (Continuous Integration / Continuous Deployment)** : Pratique consistant à automatiser les tests et la validation du code à chaque modification (CI), puis à automatiser sa livraison vers l'environnement de production (CD).
- **Lazy Loading (Chargement différé)** : Motif de conception (Design Pattern) utilisé par l'API `FastAPI`. Le fichier lourd du modèle d'IA n'est pas chargé en mémoire (RAM) au démarrage du serveur, mais uniquement lorsqu'un utilisateur effectue la première demande de prédiction, optimisant ainsi les ressources au lancement.
- **Swagger / OpenAPI** : Standard industriel définissant la structure d'une API REST. Il permet de générer automatiquement une page web interactive listant les requêtes possibles, leurs paramètres et les réponses attendues.
- **Zero Downtime** : Capacité d'une architecture logicielle à mettre à jour ses composants (comme remplacer un modèle d'IA V1 par un modèle V2) sans que l'application ne s'arrête ou ne rejette de requêtes des utilisateurs.